Hidden in the Leftovers

Task-Specific Reconstruction Gaps of Sparse Autoencoders
in English and Indian Languages

Prakhar Singhal (2022111025, prakhar.singhal@research.iiit.ac.in)
Raveesh Vyas (2022114002, raveesh.vyas@research.iiit.ac.in)

Mechanistic Interpretability of Language Models (CL3.412)
Course project proposal · Team IpCp-- · October 2026

Summary. Sparse autoencoders (SAEs) break a language model's internal activations into interpretable features, and are judged by how well they reconstruct those activations on average: 98–99% for the widely used Gemma Scope SAEs. An average can hide what is lost on a particular task. In a pilot study on gemma-2-2b we found such a loss. On an object-tracking task, the SAE's features carry only 74% of the information the model uses at layer 12, and a single direction in the SAE's leftover error, estimated on that task, restores about half of the rest; directions from other tasks or ordinary text restore almost none. We propose to map how common such task-specific gaps are, to test whether this one reflects how the model links objects to locations, and to measure whether gaps are larger in Hindi and Hinglish, since the SAEs were trained mostly on English.

1 What: research question and hypothesis

Background. Inside a language model, each token (a word or word piece) is represented at every layer by an **activation** x, a vector of 2304 numbers in gemma-2-2b. A **sparse autoencoder** (SAE) rewrites it as $x = \hat{x} + \varepsilon$: the **features** $\hat{x}$ are built from a few interpretable components, and the **SAE error** ε is whatever is left over ("dark matter" [4]). If the features captured everything the model uses, swapping only the features between two prompts would have the same effect on the answer as swapping the whole activation. The share of that effect the features fail to carry is the SAE's **gap** on the task. A **direction** is a fixed vector in activation space; Appendix E defines all terms.

Research question. *When an SAE's features miss information that a model uses for a task, is the missing part specific to that task, and is more of it missed in Indian languages?*

Hypothesis. Gaps come in two kinds. A *concentrated, task-specific* gap is largely closed by adding back the error along one or two directions estimated on that task; a *diffuse* gap needs many directions. Average reconstruction quality detects neither, and the pilot (Section 3) found both. We ask:

- **Q1. Generality.** Which kinds of task show a concentrated, task-specific gap, and is the gap tied to one prompt template or to a type of computation?
- **Q2. Mechanism.** In object tracking, is the error direction part of the model's *binding* mechanism, the way it links an object to its location [5]?
- **Q3. Language.** Are the gaps larger in Hindi and Hinglish than in English, and do the same directions close them?

2 Why: motivation

SAE suites such as Gemma Scope [9] are used to explain, audit and steer models. They are evaluated mainly by **reconstruction fidelity**, the share of the model's next-token performance kept when $\hat{x}$ replaces x; we measure 98–99% for the SAEs we use. Fidelity is an average over ordinary text, so it cannot show what is lost on a particular computation, yet the SAE error is sizeable, partly predictable from the input [4], and matters inside feature circuits [11].

The answer decides how SAEs should be used. If what they miss is *generic*, one fix (a wider SAE, or a correction fitted on ordinary text) serves every task. If it is *task-specific*, a feature-based explanation of a task can be incomplete exactly where fidelity does not look. Language raises the same question: the Gemma Scope SAEs were trained mostly on English text [9], and prior work finds language-specific SAE features [1, 2] but does not measure what the error carries in each language.

Table 1: The four pilot tasks, with one real pair each. Prompt B changes one step of prompt A.

Task	Prompt A (answer)	Change in prompt B (answer)
Running sum	`Start with 2. Add 3. Add 1. Add 2. Total =` (8)	`Add 1` becomes `Add 3` (10)
Object tracking [8]	`The pen is in box B. The coin is in box G. The key is in box C. The coin moves to box A. The pen moves to box G. The coin is in box` (A)	the second move is made by the coin (G)
Variable copying	`y = 4. r = 4. x = r. r = 3. x =` (4)	`r = 4` becomes `r = 7` (7)
Names (IOI) [14]	`When Lucy and David went to the river, David gave a drink to` (Lucy)	Lucy is the giver (David)

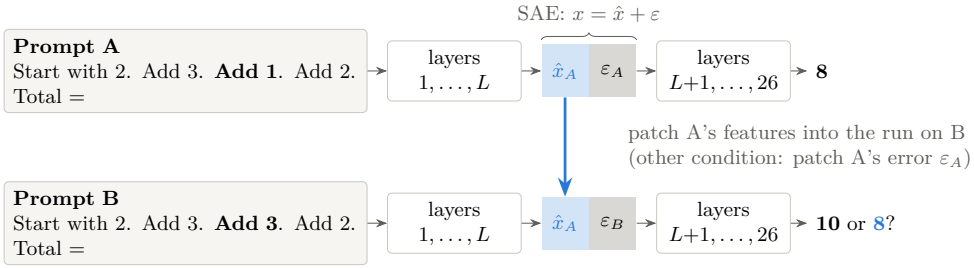


Figure 1: Activation patching with an SAE. At layer L the SAE splits the activation into features $\hat{x}$ and error ε. If A's features alone move B's answer from 10 to 8, the features carry the task's information.

3 What has been done: pilot study

Setup. We used gemma-2-2b, an open 2-billion-parameter model [6], with Gemma Scope SAEs of 16k to 262k features (Appendix B.1). Each task is a set of prompt pairs that differ in one step, so that their answers differ (Table 1). All reported numbers are on held-out pairs. The names task (IOI) is a well-studied control rather than a tracking task. Appendix A explains how these tasks replaced the original ones.

Method: activation patching [15] (Figure 1). We run the model on prompt B and, at one layer, replace one part of its activation ($\hat{x}$ or ε) with the same part from prompt A. The **effect recovered** is how far this moves the model's answer from B's towards A's: 0% means no change, 100% means as far as replacing the whole activation. The gap is 100% minus the effect recovered by the features alone.

Findings (details in Appendix C):

1. **The features carry most of what the model uses.** On the three tracking tasks, patching the features recovers 74–100% of the effect at every layer tested. The error alone recovers little, and most of what it does recover is matched by simply replacing it with random noise.
2. **One gap is concentrated and task-specific.** For object tracking at layer 12 the features recover 74%, a gap of 26%. Adding back A's error along one direction estimated on this task raises this to 88%, closing about half of the gap (Figure 2). Directions from other tasks, ordinary text or random choice close at most 2%. The running sum at layer 18 behaves the same way on a smaller gap (13%).
3. **Another gap is diffuse.** The names task loses 24–36% at layers 18–20, but no small set of directions accounts for it: 16 to 32 directions are needed to close half.
4. **Wider SAEs shrink the gap but do not remove it.** From 16k to 262k features the object-tracking gap falls from 26% to 15%, and one error direction still closes more than half of it (width and sparsity are not yet separated).

Three measurement pitfalls that would have misled us, and how the pipeline avoids them, are in Appendix C.5.

Limitations. One model, one SAE family, synthetic tasks, and width varied at one layer only. Patching along a few directions can create effects the model does not normally use [10]; our random and cross-task controls address this only in part.

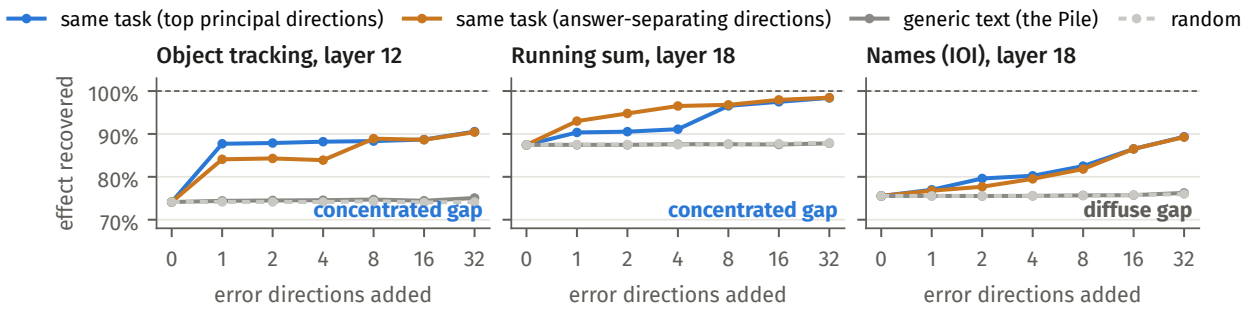


Figure 2: Concentrated and diffuse gaps: effect recovered when A's features are patched together with A's error along N directions. Same-task directions close much of the gap after one or two directions (left, middle); generic-text and random directions close nothing; the names gap closes only slowly (right).

4 How: plan and success criteria

Q1, generality. We group tasks by the computation they need: *entity binding* (objects in boxes; people and their cities, e.g. "Asha lives in Pune. Ravi lives in Delhi. Asha lives in"), *counting* (longer running sums), *reference following* (variable copying) and the names control. For each task, layer and SAE width we measure the gap, how many directions close half of it, and a **specificity score**: the share closed by same-task directions minus the largest share from any other source. If directions transfer between the two binding tasks, the gap belongs to the computation rather than to one prompt template; SAEs that use equally many features per token separate width from sparsity.

Q2, mechanism. Feng and Steinhardt [5] found that LLaMA and Pythia models tag each entity and its attribute with a shared "binding ID" vector. We will estimate these vectors for gemma-2-2b, compare them with the object-tracking error direction, and shift activations along that direction to test whether the answer moves to the other box.

Q3, language. We will run the same tasks in Hindi (Devanagari), Hinglish (Latin script mixed with English words), Hindi in Latin script without English words, Tamil or Bengali if the model can do the tasks in them, and French as a non-Indian control. Example, the running sum in Hindi: शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल $= \rightarrow$ 8 (design in Appendix D).

Table 2: Success criteria, fixed in advance (95% confidence intervals; otherwise inconclusive).

Question	Supported if	Not supported if
Q1	gap $\geq$ 10%, and same-task directions close $\geq$ 30 points more than any other source; transfer between binding tasks $\geq$ half of own	on $\geq$ 2 new tasks with a gap, other sources close as much as same-task directions
Q2	shifting along the direction moves $\geq$ 50% of pairs to the other box; a random direction $\leq$ 5%	the shift is no more effective than a random direction
Q3	Hindi gap larger than English, with non-overlapping intervals (fidelity and transfer criteria in Table D.1)	Hindi gap not larger

Deliverables: (1) a map of gap size, type and specificity across tasks, layers, widths and languages; (2) the patching code, with the three pitfalls handled; (3) a tested answer to the binding hypothesis. **Resources:** one 8 GB GPU and the working pilot pipeline; risks and fallbacks in Appendix D.

5 When: timeline

Table 3: Timeline. Each half-week ends with a checkpoint.

Dates	Work	Checkpoint
Oct 19–26	Pilot study (done; Appendix B.6)	concentrated gap found
Oct 27–29	Q2: binding vectors and the shifting test; Q1: new tasks	Q2 result
Oct 30–Nov 2	Q1: matched-sparsity widths; Q3: task performance in Hindi, Hinglish	usable languages
Nov 3–9	Q1: gap and specificity per task; Q3: fidelity, gap, transfer	Q1 and Q3 tables
Nov 10–12	Buffer; more test pairs; robustness checks	final figures
Nov 13–16	Report and final presentation	submission

References

[1] J. Brinkmann et al. Large language models share representations of latent grammatical concepts across typologically diverse languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6131–6150, Apr. 2025. Association for Computational Linguistics.

[2] B. Deng et al. Unveiling language-specific features in large language models via sparse autoencoders. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4563–4608, July 2025. Association for Computational Linguistics.

[3] S. Doddapaneni et al. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12402–12426, July 2023. Association for Computational Linguistics.

[4] J. Engels, L. R. Smith, and M. Tegmark. Decomposing the dark matter of sparse autoencoders. *Transactions on Machine Learning Research*, 2025.

[5] J. Feng and J. Steinhardt. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[6] Gemma Team et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[7] J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Nov. 2019. Association for Computational Linguistics.

[8] N. Kim and S. Schuster. Entity tracking in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855, July 2023. Association for Computational Linguistics.

[9] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Nov. 2024. Association for Computational Linguistics.

[10] A. Makelov et al. Is this the subspace you are looking for? An interpretability illusion for subspace activation patching. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[11] S. Marks et al. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[12] G. Penedo et al. FineWeb2: One pipeline to scale them all — adapting pre-training data processing to every language. In *Second Conference on Language Modeling (COLM)*, 2025.

[13] S. Rajamanoharan et al. Jumping ahead: Improving reconstruction fidelity with JumpReLU sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.

[14] K. R. Wang et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[15] F. Zhang and N. Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Appendices

The original plan, the pilot methods and every pilot result

A From the original plan to the pilot tasks
B Pilot methods: model and SAEs, tasks, patching, repair, probes, runs
C Pilot results: patching, repair, specificity, width, pitfalls, replication, verdict, caveats
D Design of the planned study
E Glossary

A From the original plan to the pilot tasks

A.1 Original hypotheses

The project began as *PRISM* (Probing Residual Information and State Manifolds), a test of a strong version of the "dark matter" hypothesis: that SAEs fail to represent the step-by-step state a model computes, so that this state resides in the SAE error. It made three hypotheses, each tied to one experiment (Table A.1).

Table A.1: The original hypotheses and the experiment designed to test each.

	Hypothesis	Experiment	Prediction if true
H1	The SAE error carries the task state	Patch $\hat{x}$ or ε from prompt A into the run on prompt B	patching ε recovers more than patching $\hat{x}$
H2	The state forms a low-dimensional structure in the error	Spread of ε across directions; probes for the state; low-dimensional projections	a probe reads the state from ε better than from $\hat{x}$ and better than from the prompt's words
H3	A few error directions repair the SAE	Features plus A's error along its top r directions	a small r closes most of the gap; random directions do not

A.2 Original tasks

The plan used three algorithmic tasks whose answer depends on a hidden running state: a sum with a remainder (modular arithmetic), the depth of nested brackets, and the size of the graph reachable from a node, with the names task as a control. Experiments were planned at layers 4–24 and SAE widths from 16k upwards.

A.3 First run and code checks

The first full run (October 19) produced no usable results: too few prompt pairs survived the filter that keeps only pairs the model solves. A code review then found pipeline faults as well as model limits; for example, the worked examples meant to precede each prompt were not being added. These were fixed before the pilot. The same review confirmed that our model implementation agrees with the reference (Hugging Face) implementation: run in 16-bit precision, the two give the same top answer on 95% of test prompts.

A.4 Capability study

We then measured each task on 128 pairs per setting, varying the number of worked examples (0–8) and the difficulty. A setting was usable if the model answered at least 60% of prompts correctly, at least 30% of pairs passed the pair filter, and the model beat the best shortcut (such as repeating the starting number) by a clear margin (the lower end of the 95% interval). The three original tasks failed at every setting (Table A.2); for bracket depth, most wrong answers simply repeated the starting depth. We therefore replaced them with tasks the model can do, keeping the same question.

Table A.2: Capability study. Original tasks at their planned difficulty; replacement tasks at the setting used in the pilot runs ("kept in run" is the share of generated pairs that passed the pilot's pair filter).

Task	Setting	Top answer correct	Best shortcut	Kept in run
Sum with remainder	4 steps, best of 0–8 examples	13%	21%	–
Bracket depth	8 operations, best of 0–8 examples	41%	41%	–
Graph size	4 edges, best of 0–8 examples	30%	48%	–
Running sum	3 steps, 8 examples	98%	26%	62%
Object tracking	2 moves, 8 examples	98%	65%	27%
Variable copying	1 hop, 2 examples	22%	16%	40%
Names (IOI)	2 examples	100%	17%	100%

Variable copying is weak with 2 examples (it reaches 95% with 8); the pilot kept 2 examples but drew from a larger pool of candidates, which still left enough pairs the model solves.

B Pilot methods

B.1 Model and SAEs

Table B.1: Model, SAEs and their quality.

Item	Value
Model	gemma-2-2b: 26 layers, 2304 numbers per token, run in 16-bit (bf16) precision
SAEs	Gemma Scope residual-stream SAEs with a JumpReLU activation [9, 13] (`gemma-scope-2b-pt-res-canonical`); widths 16k, 32k, 65k, 131k and 262k
Fidelity (16k)	next-token performance kept on generic text: 99.1% (layer 10), 98.7% (12), 98.5% (14), 98.0% (18), relative to replacing the activation by its average; 3936 tokens of the Pile
Error size (16k)	$\|\varepsilon\|/\|x\|$ on the pilot prompts: 0.08–0.09 at layer 4, 0.12–0.18 at layers 8–16, 0.11–0.13 at layer 20
Compute	one 8 GB laptop GPU (RTX 5060)

B.2 Tasks

Each task generator produces pairs whose prompts have the same number of tokens and whose answers are single tokens; pairs that break this are dropped and counted. Worked examples are prepended identically to both prompts. The **pair filter** keeps only pairs the model handles: *strict* requires the model's top answer to be correct on both prompts and its preference for A's answer over B's to exceed 2 logits on prompt A; *clean-only* requires a correct top answer on prompt A and a change in preference of more than 2 logits between the prompts. A task is skipped if fewer than 20 pairs survive.

Table B.2: Task settings in the main pilot. "Kept" is the share of generated pairs that passed the filter; chance is one over the number of possible answers. Each task has 300 training and 150 test pairs.

Task	Difficulty	Worked examples	Filter	Kept	Chance
Running sum	3 steps	8	strict	62%	25%
Object tracking	3 objects, 2 moves	8	clean-only	27%	18%
Variable copying	1 copy hop	2	strict	40%	11%
Names (IOI)	–	2	strict	100%	11%

The running sum also uses a story form ("Anna has 5 cards. Anna gets 1 more. Anna gets 2 more. Anna gets 1 more. Anna now has" → 9; in B the second step becomes "gets 1 more" → 8). Variable copying always opens with a distracting assignment, so that "the first number" is not a shortcut.

B.3 Activation patching

Effect recovered (NLDR, normalized logit-difference recovery). A *logit* is the model's raw score for a possible next token. With D the logit of A's answer minus the logit of B's answer, NLDR =

$(D_{\text{patched}} - D_B)/(D_A - D_B)$, averaged over test pairs: 0 means no change, 1 means the same as patching the whole activation. Values can exceed 1. Intervals are 95% paired bootstrap intervals (1000 resamples).

Conditions (all at one layer; the subscript names the prompt each part comes from):

Features only, error only, full activation
$\hat{x}_A + \varepsilon_B$; $\hat{x}_B + \varepsilon_A$; and x_A, which gives 1 by construction.

Noise control
$\hat{x}_B + \eta$, with random noise η of the same size as ε: it erases B's error without supplying A's.

Features alone
$\hat{x}_A$ with no error at all.

Error resampling
(last token only) $\hat{x}_B$ plus the error of another pair whose prompt A has the same answer, or a different answer; and $\hat{x}_B$ plus the average training error. If the error carried the answer, same-answer resampling would beat different-answer resampling.

Positions. Every result patches all tokens from the first one where A and B differ, except pitfall 1, which patches the last token only (Figure B.1).

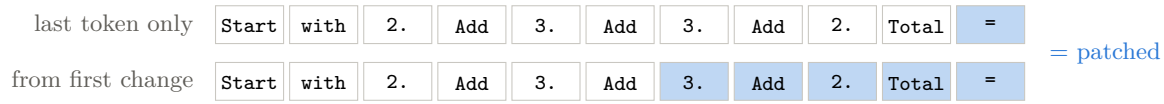


Figure B.1: Patch positions for a running-sum pair. Earlier tokens are identical in A and B, so patching from the first changed token onwards covers everything that differs; the last token alone may not yet hold the state.

Decision rules, fixed before the run. H1 is judged only at layers where patching the full activation recovers at least 0.8. A gap counts as persistent across widths if the widest SAE keeps at least 60% of the 16k gap.

B.4 Repair with error directions

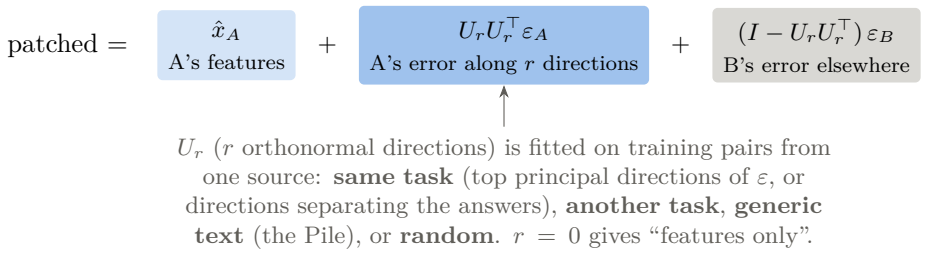


Figure B.2: Repair. The run on B receives A's features, A's error along r chosen directions, and B's own error in all other directions.

Directions are estimated on the 300 training pairs only: *top principal directions* are the directions in which the task's training errors vary most; *answer-separating directions* are the principal directions of the differences between the average errors for each answer. Other sources: the same estimate on another task (*transfer*), on generic text (the Pile, 40–100 documents of 128 tokens), or random directions (2–3 seeds). We test $r = 0, 1, 2, 4, 8, 16, 32$ (and 64 in the audit run). The **share of the gap closed** is (repair − features only) / gap.

B.5 Probes

A probe is a linear classifier trained on the training pairs to read the answer from the activation at the last token, and scored on the test pairs. We probe x, $\hat{x}$, ε and the SAE's feature activations (2048 columns), report accuracy and selectivity (accuracy minus the accuracy with shuffled labels), and compare with the same probe on the prompt's tokens alone.

B.6 Runs, compute and failures

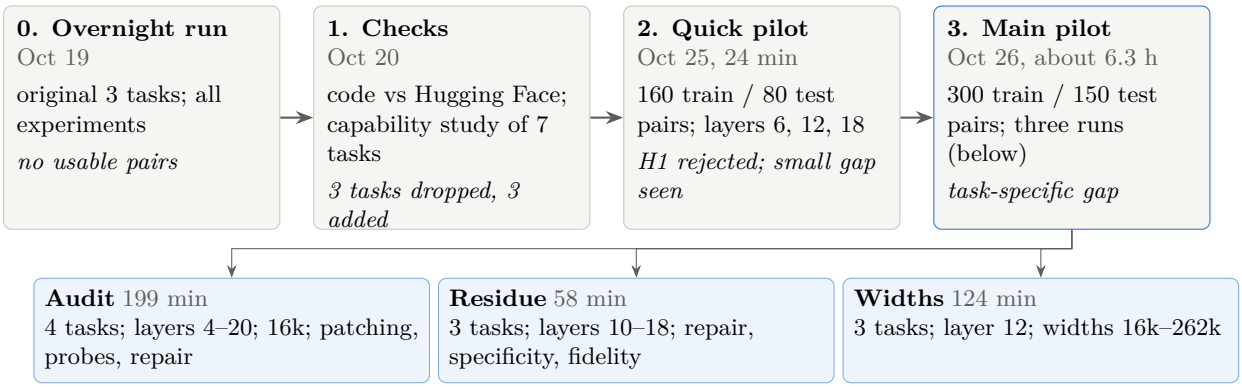


Figure B.3: The four pilot stages. The main pilot consisted of three runs on one 8 GB GPU.

Table B.3: Run inventory (all with 16k SAEs unless stated).

Run	Tasks and layers	Experiments	Time
Quick pilot	4 tasks; layers 6, 12, 18; 160/80 pairs	patching (both positions, controls); repair at 12, 18	24 min
Audit	4 tasks; layers 4, 8, 12, 16, 20	patching (both positions, controls); probes at 8–20; repair up to 64 directions	199 min
Residue	3 tasks (no variable copying); layers 10, 12, 14, 18	patching; repair with generic-text and transfer directions; fidelity	58 min
Widths	3 tasks; layer 12; widths 16k–262k	patching; repair; fidelity	124 min

Two cases have no repair results: layer 10 in the residue run (direction estimation failed numerically) and the names task at 65k in the widths run (out of GPU memory).

C Pilot results

C.1 Features versus error, by layer

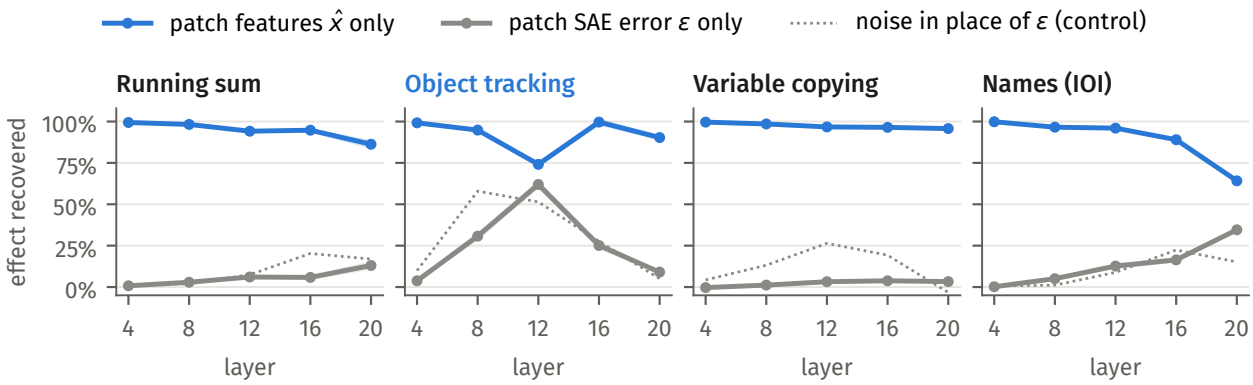


Figure C.1: The features carry most of the effect at every layer; the error alone carries little. Shaded bands are 95% intervals. Object tracking's error curve at layers 8–16 follows the noise control, so most of it comes from erasing B's error rather than from A's error.

Table C.1: Effect recovered (%) by patching the features / the error, per layer.

Task	Layer 4	Layer 8	Layer 12	Layer 16	Layer 20
Running sum	99 / 1	98 / 3	94 / 6	95 / 6	86 / 13
Object tracking	99 / 4	95 / 31	74 / 62	100 / 25	90 / 9
Variable copying	100 / 0	99 / 1	97 / 3	96 / 4	96 / 3
Names (IOI)	100 / 0	97 / 5	96 / 13	89 / 16	64 / 35

Controls at layer 12. Replacing B's error with random noise recovers 7, 51, 26 and 9% for the four tasks (in the order of Table C.1), so for object tracking most of the error's 62% comes from erasing B's error rather than from supplying A's. Using A's features with no error at all recovers 102, 109, 77 and 97%. For object tracking, an error taken from a different pair works about as well whether that pair's answer matches A's or not (28 vs 26%, last token only): the error's effect does not depend on the answer, which argues against H1.

C.2 Repair

Table C.2: Repair for every case with a gap of at least 5% (layers 8–20). Share of the gap closed (%) by one and two directions (top principal / answer-separating), by two generic-text and two random directions, and the number of directions needed to close half of the gap. Variable copying has no gap of 5% or more.

Task	Layer	Gap	1 direction	2 directions	Generic (2)	Random (2)	For half
Running sum	12	6	−2 / 9	−4 / 18	0	1	>32 / >32
	14	6	−1 / 14	−5 / 22	−1	2	>32 / >32
	16	5	−4 / 36	6 / 46	–	−1	32 / 4
	18	13	23 / 44	25 / **58**	0	1	8 / **2**
	20	14	19 / 14	23 / 43	–	0	8 / 4
Object tracking	8	5	−16 / −8	−26 / 0	–	0	>32 / >32
	12	26	**52** / 38	53 / 39	1	0	**1** / 8
	18	14	−2 / 26	18 / 36	2	1	16 / 4
	20	10	−32 / 29	−6 / 47	–	−2	16 / 4
Names (IOI)	14	10	1 / 2	1 / 2	1	0	>32 / >32
	16	11	−1 / −3	−13 / −6	–	0	>32 / >32
	18	24	6 / 5	17 / 9	0	0	32 / 32
	20	36	10 / 9	19 / 15	–	0	16 / 16

Only two cases are concentrated in the sense of the hypothesis (one or two directions close half the gap): object tracking at layer 12 and the running sum at layer 18. The other gaps need 4 to more than 32 directions, and small gaps give noisy, sometimes negative, shares. Generic-text directions were tested only in the residue run (layers 12, 14 and 18; "–" elsewhere).

C.3 Task specificity

Table C.3: Share of the gap closed (%) by two directions from each source.

Task (gap)	Source	Share closed
Object tracking, layer 12 (26%)	same task: top principal / answer-separating	53 / 39
	other task: names / running sum	2 / −1
	generic text / random	1 / 0
Running sum, layer 18 (13%)	same task: top principal / answer-separating	25 / 58
	other task: object tracking / names	1 / 0
	generic text / random	0 / 1

C.4 Width

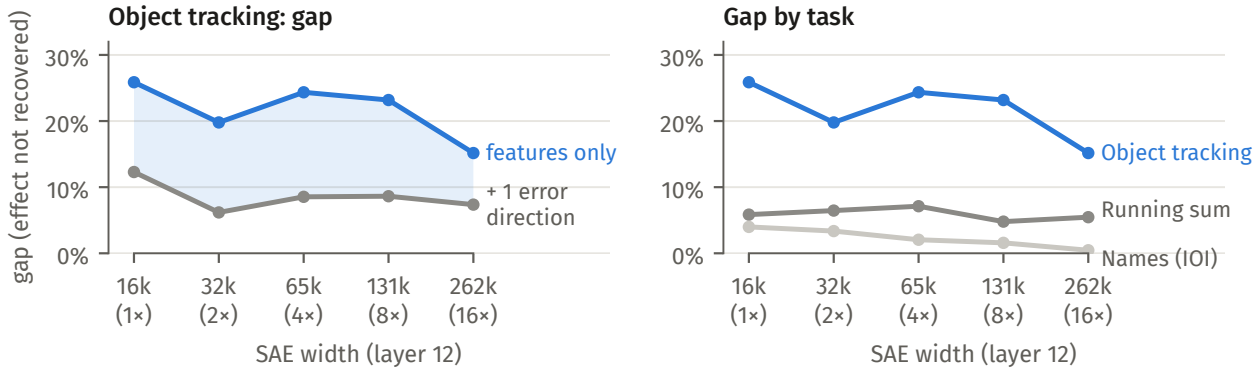


Figure C.2: Wider SAEs at layer 12. Left: the object-tracking gap with the features only and with one error direction added; the shaded band is what that direction closes. Right: the gap for three tasks.

Table C.4: Width sweep at layer 12 (%). "+1 error" and "+1 random" add one top principal or random direction; "active" is the mean number of features switched on per token.

Task	Width	Features only [95% CI]	Gap	+1 error	+1 random	Active
Object tracking	16k	74 [73, 76]	26	88	74	78
	32k	80 [79, 82]	20	94	80	75
	65k	76 [74, 77]	24	91	76	76
	131k	77 [75, 78]	23	91	77	131
	262k	85 [84, 86]	15	93	85	124
Running sum	16k	94 [93, 95]	6	94	94	80
	32k	94 [92, 95]	6	94	94	80
	65k	93 [91, 94]	7	93	93	75
	131k	95 [94, 96]	5	95	95	131
	262k	95 [93, 96]	5	94	95	129
Names (IOI)	16k	96 [95, 97]	4	96	96	89
	32k	97 [96, 98]	3	97	97	82
	65k	98 [97, 99]	2	–	–	82
	131k	98 [98, 99]	2	98	98	138
	262k	100 [99, 100]	0	100	100	139

The 262k SAE keeps 15.2% of the 25.9% object-tracking gap (59%), just under the 60% persistence rule (Appendix B.3). The 131k and 262k SAEs also switch on more features per token than the narrower ones, so part of the drop may come from sparsity rather than width.

C.5 Three pitfalls

Pitfall 1: patching only the last token. In early and middle layers the state still sits on earlier tokens, so patching only the last token recovers little even with the whole activation (Table C.5); patching from the first changed token recovers 100% at every task and layer [15].

Table C.5: Effect recovered (%) when the whole activation is patched at the last token only.

Task	Layer 4	Layer 8	Layer 12	Layer 16	Layer 20
Running sum	0	0	3	58	94
Object tracking	1	4	47	77	85
Variable copying	0	0	0	5	55
Names (IOI)	0	0	0	60	81

Pitfall 2: the prompt gives the answer away. A probe on the prompt's tokens alone scores 100%

on variable copying and names (Table C.6), so a high probe score there says nothing about the model's internal state [7]. **Pitfall 3: readable is not used.** For object tracking and names, probes read the answer far better from ε than from $\hat{x}$ (84 vs 48%; 97 vs 20%), yet patching that token recovers at most 47%: the model is not yet using the information at that position.

Table C.6: Probe accuracy at the last token, layer 12 (%), and the effect of patching that token.

Task	Probe on x	On ε	On $\hat{x}$	Prompt tokens only	Chance	Last-token patch
Running sum	57	53	42	45	25	3
Object tracking	91	84	48	33	18	47
Variable copying	98	95	39	100	11	0
Names (IOI)	100	97	20	100	11	0

C.6 Replication: quick pilot versus main pilot

Table C.7: The quick pilot (160/80 pairs) and the main pilot (300/150 pairs) agree.

Quantity	Quick pilot	Main pilot
Object tracking, layer 12: features only / error only / noise control	75 / 61 / 50	74 / 62 / 51
Object tracking, layer 12: share of gap closed by one direction	51	52
Running sum, layer 18: gap	12	13
Names, layer 18: gap; share closed by 32 directions	24; 55	24; 56

C.7 Verdict on the original hypotheses

Table C.8: What the pilot says about each original hypothesis.

Hypothesis	Verdict
H1 (error carries the state)	**Rejected** for every task: $\hat{x}$ recovers more than ε at every layer, and the error's effect does not depend on which answer it came from.
H2 (low-dimensional structure in the error)	**Not supported as stated**: the answer can be read from ε but is not used there, and for two tasks the prompt's words alone give it away.
H3 (a few directions repair)	**Supported, narrowly**: one direction closes about half of object tracking's gap at layer 12, and two close 58% of the running sum's at layer 18; the directions are task-specific. This became the project's hypothesis.

C.8 Caveats

- One model (gemma-2-2b), one SAE family (Gemma Scope) and synthetic tasks; width was tested at layer 12 only.
- Object tracking is the hardest task for the model: only 27% of generated pairs passed its filter, so its test pairs are the most heavily selected and may over-represent easier cases.
- The names control's late-layer loss (24–36% at layers 18–20) is spread over many directions: part of the late-layer loss is a general SAE weakness rather than a task-specific gap.
- Width and sparsity are confounded in the Gemma Scope family (Table C.4).
- The binding explanation is a hypothesis, and the multilingual work is a plan with no results yet.

D Design of the planned study

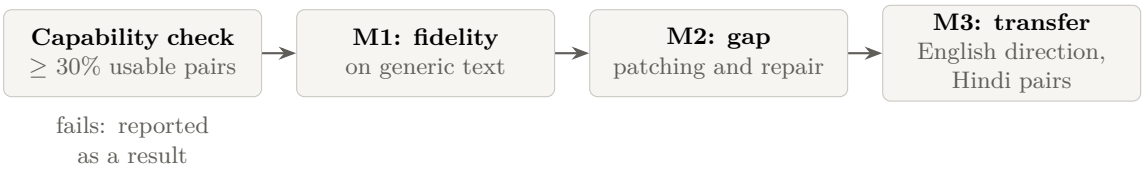


Figure D.1: Steps for each language. A language passes on only if the model can do the tasks in it.

Table D.1: Multilingual measures and criteria (95% intervals; otherwise inconclusive).

ID	Measure	Supported if	Not supported if
M1	fidelity on generic text, per language	Hindi ≥ 3 points below English	within 1 point of English
M2	gap and specificity on the tasks	Hindi gap larger, non-overlapping intervals	Hindi gap not larger
M3	English direction applied to Hindi pairs	closes ≥ half as much as Hindi's own direction	closes under a quarter as much

Languages. English (baseline); Hindi in Devanagari (the best-resourced Indian language, most likely to work); the same Hindi in Latin script without English words, and Hinglish (Latin script, mixed with English words), which together separate the effect of script from that of vocabulary; Tamil or Bengali if the model can do the tasks; French as a high-resource non-Indian control. Q2 (binding) is run in English only.

Templates. The answer must be the last token. Arithmetic templates can end with "=" as in English; templates that need a verb at the end in Hindi use a question followed by "answer:". Examples (single-token answers are checked against the tokenizer before use):

- Object tracking (Hindi): सिक्का डिब्बे B में है। चाबी डिब्बे E में है। चाबी डिब्बे C में चली जाती है। चाबी किस डिब्बे में है? डिब्बा → C
- Names (Hinglish): `Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` → Rahul
- Variable copying is code-like and identical in every language: a language-neutral control.

Constraints. Prompts A and B need the same number of tokens and a one-token answer; many Hindi names split into several tokens, so names are chosen from single-token ones (the pipeline already drops and counts pairs that break this rule). **Data.** Fidelity per language uses generic text from IndicCorp v2 [3] and FineWeb2 [12].

Risks and fallbacks.

- Binding vectors may not be estimable for gemma-2-2b; Q2 then keeps only the shifting test.
- The model may not do the tasks in Tamil or Bengali; we report this and keep Hindi and Hinglish.
- Names that split into several tokens are replaced by letters or digits as answers.
- Width results at one layer may not generalise; Q1 repeats them at a second layer if time allows.

E Glossary

Table E.1: Terms used in this proposal.

Term	Meaning
Token	a word or word piece; the unit the model reads
Layer	one of gemma-2-2b's 26 processing stages
Activation x	the model's internal vector for one token at one layer (2304 numbers; also "residual stream")
Sparse autoencoder (SAE)	a tool that rewrites an activation as a few active, interpretable features
Features $\hat{x}$	the SAE's reconstruction of x from its active features
SAE error ε	$x - \hat{x}$: what the SAE does not explain ("dark matter"; informally, the leftovers)
Width	how many features an SAE can choose from (16k = 16,384, up to 262k)
Active features	the number of features switched on for one token (also L0)
Reconstruction fidelity	share of next-token performance kept when $\hat{x}$ replaces x, relative to replacing x by its average (cross-entropy loss recovered)
Prompts A and B	two prompts that differ in one step, so the right answer differs (also "clean" and "corrupt")
Activation patching	running the model on B while replacing part of its activation with A's at one layer
Logit	the model's raw score for a possible next token
Effect recovered (NLDR)	how far patching moves the model's preference from B's answer to A's, from 0 to 1
Gap	the effect the features fail to carry: one minus the effect recovered by patching the features only
Direction	a fixed vector in the 2304-dimensional activation space
Top principal directions	the directions in which a set of errors varies most
Answer-separating directions	directions that best separate the average errors for different answers
Specificity score	share of the gap closed by same-task directions minus the largest share from any other source
Probe	a simple linear classifier trained to read a value, such as the box, from an activation
Binding	how a model links an entity to its attribute, such as an object to its box
IOI	"indirect object identification", the names task [14]
Generic text	ordinary text from the Pile, used for fidelity and for generic-text directions
Hinglish	Hindi written in Latin script and mixed with English words
95% CI	bootstrap confidence interval: the range the result would likely fall in on new test pairs